Rake_anthony_p12_notebook1_v3_acp

January 13, 2026

Détectez des faux billets avec Python

Etape 1 - Importation des librairies et chargement des fichiers

[3]:
```
import pandas as pd
import numpy as np

import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    confusion_matrix, classification_report, ConfusionMatrixDisplay
)

from sklearn.linear_model import LogisticRegression
from sklearn.cluster import KMeans
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
```

[4]:
```
df = pd.read_csv("billets_parametrage.csv", sep=';')
df.head()
```

[4]:
```
   is_genuine  diagonal  height_left  height_right  margin_low  margin_up  \
0        True    171.81       104.86        104.95        4.52       2.89
1        True    171.46       103.36        103.66        3.77       2.99
2        True    172.69       104.48        103.50        4.40       2.94
3        True    171.36       103.91        103.94        3.62       3.01
4        True    171.73       104.28        103.46        4.04       3.48

   length
0  112.83
1  113.09
2  113.16
3  113.51
4  112.54
```

Etape 2 - Analyse exploratoire

2.1 - Caractéristiques du df

[7]:
```
print(df.info())
print(df.describe())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1500 entries, 0 to 1499
Data columns (total 7 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   is_genuine    1500 non-null   bool
 1   diagonal      1500 non-null   float64
 2   height_left   1500 non-null   float64
 3   height_right  1500 non-null   float64
 4   margin_low    1463 non-null   float64
 5   margin_up     1500 non-null   float64
 6   length        1500 non-null   float64
dtypes: bool(1), float64(6)
memory usage: 71.9 KB
None
          diagonal  height_left  height_right   margin_low    margin_up  \
count  1500.000000  1500.000000   1500.000000  1463.000000  1500.000000
mean    171.958440   104.029533    103.920307     4.485967     3.151473
std       0.305195     0.299462      0.325627     0.663813     0.231813
min     171.040000   103.140000    102.820000     2.980000     2.270000
25%     171.750000   103.820000    103.710000     4.015000     2.990000
50%     171.960000   104.040000    103.920000     4.310000     3.140000
75%     172.170000   104.230000    104.150000     4.870000     3.310000
max     173.010000   104.880000    104.950000     6.900000     3.910000

           length
count  1500.00000
mean    112.67850
std       0.87273
min     109.49000
25%     112.03000
50%     112.96000
75%     113.34000
max     114.44000
```

- les dimensions géométriques principales (longueur, hauteur gauche/droite, diagonale) sont epeu dispersées, avec un écart-type autour de 0.3 mm
- la variable margin_low présente une variabilité nettement plus élevée (écart-type : 0.66 mm) + valeurs manquantes. Son minimum (2.98 mm) et surtout son maximum (6.90 mm) suggèrent la présence de billets atypiques

[9]:
```python
# Répartition vrais/faux
print(df['is_genuine'].value_counts())
print(df['is_genuine'].value_counts(normalize=True))
```

```
is_genuine
True     1000
False     500
Name: count, dtype: int64
is_genuine
True     0.666667
False    0.333333
Name: proportion, dtype: float64
```

[10]:
```python
# Valeurs manquantes dans margin_low
df.isnull().sum()
```

[10]:
```
is_genuine       0
diagonal         0
height_left      0
height_right     0
margin_low      37
margin_up        0
length           0
dtype: int64
```

[11]:
```python
df.groupby('is_genuine')['margin_low'].apply(lambda x: x.isnull().sum())
```

[11]:
```
is_genuine
False     8
True     29
Name: margin_low, dtype: int64
```

[12]:
```python
# Vrais billets
df[df['is_genuine'] == True].describe()
```

[12]:
```
          diagonal  height_left  height_right  margin_low   margin_up  \
count  1000.000000  1000.000000    1000.00000  971.000000  1000.00000
mean    171.987080   103.949130     103.80865    4.116097     3.05213
std       0.300441     0.300231       0.29157    0.319124     0.18634
min     171.040000   103.140000     102.82000    2.980000     2.27000
25%     171.790000   103.740000     103.61000    3.905000     2.93000
50%     171.990000   103.950000     103.81000    4.110000     3.05000
75%     172.200000   104.140000     104.00000    4.340000     3.18000
max     172.920000   104.860000     104.95000    5.040000     3.74000

            length
count  1000.000000
mean    113.202430
```

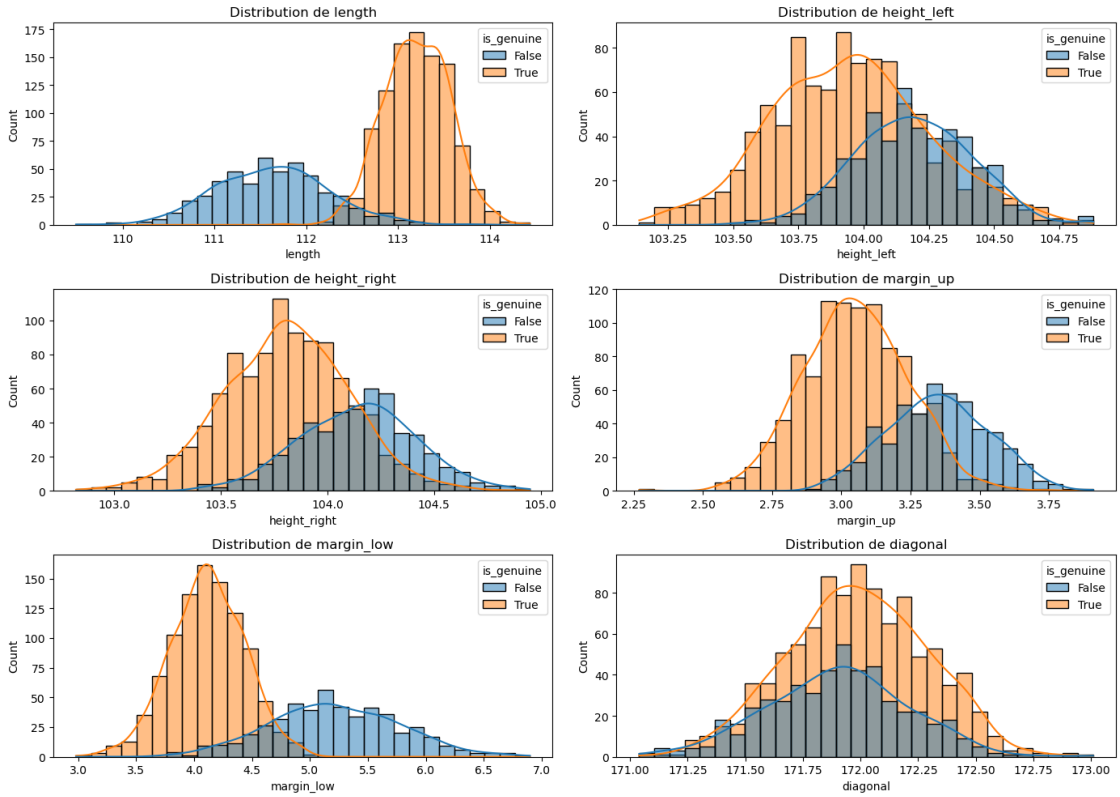
```
std       0.359552
min     111.760000
25%     112.950000
50%     113.205000
75%     113.460000
max     114.440000
```

[13]:
```python
# Faux billets
df[df['is_genuine'] == False].describe()
```

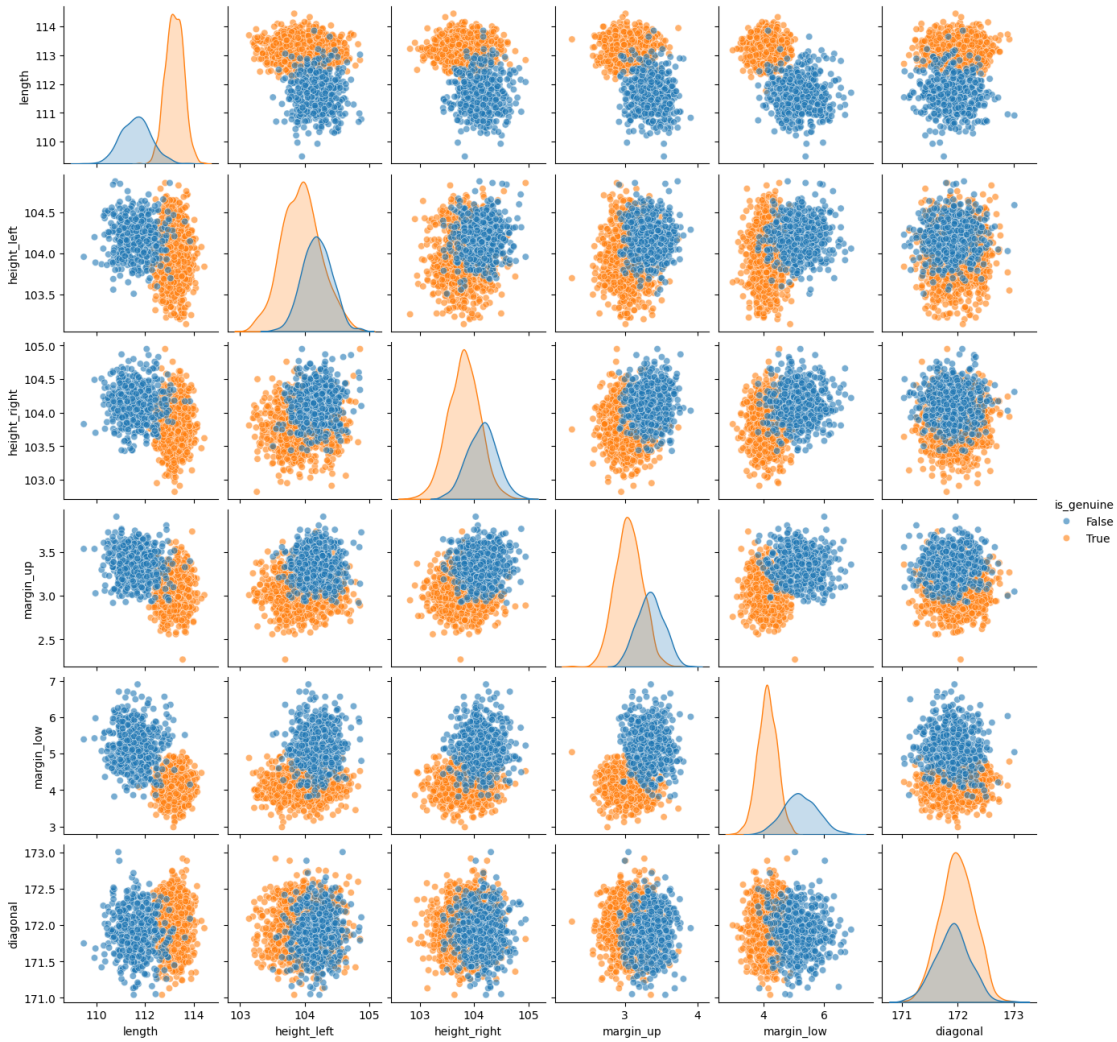
[13]:
```
         diagonal  height_left  height_right  margin_low   margin_up  \
count  500.000000   500.000000    500.000000  492.000000  500.000000
mean   171.901160   104.190340    104.143620    5.215935    3.350160
std      0.306861     0.223758      0.270878    0.553531    0.180498
min    171.040000   103.510000    103.430000    3.820000    2.920000
25%    171.690000   104.040000    103.950000    4.840000    3.220000
50%    171.910000   104.180000    104.160000    5.190000    3.350000
75%    172.092500   104.332500    104.320000    5.592500    3.472500
max    173.010000   104.880000    104.950000    6.900000    3.910000

           length
count  500.000000
mean   111.630640
std      0.615543
min    109.490000
25%    111.200000
50%    111.630000
75%    112.030000
max    113.850000
```

2.2 - Analyse graphique

[15]:
```python
# Vérification de la cohérence et de la distribution avant imputation
import matplotlib.pyplot as plt
import seaborn as sns

features = ['length', 'height_left', 'height_right',
            'margin_up', 'margin_low', 'diagonal']

plt.figure(figsize=(14,10))
for i, col in enumerate(features, 1):
    plt.subplot(3, 2, i)
    sns.histplot(data=df, x=col, hue='is_genuine', kde=True, bins=30)
    plt.title(f"Distribution de {col}")
plt.tight_layout()
plt.show()
```



[16]:
```
sns.pairplot(df, vars=features, hue='is_genuine',
             plot_kws={'alpha':0.6}, diag_kind='kde', height=2.2)
plt.show()
```



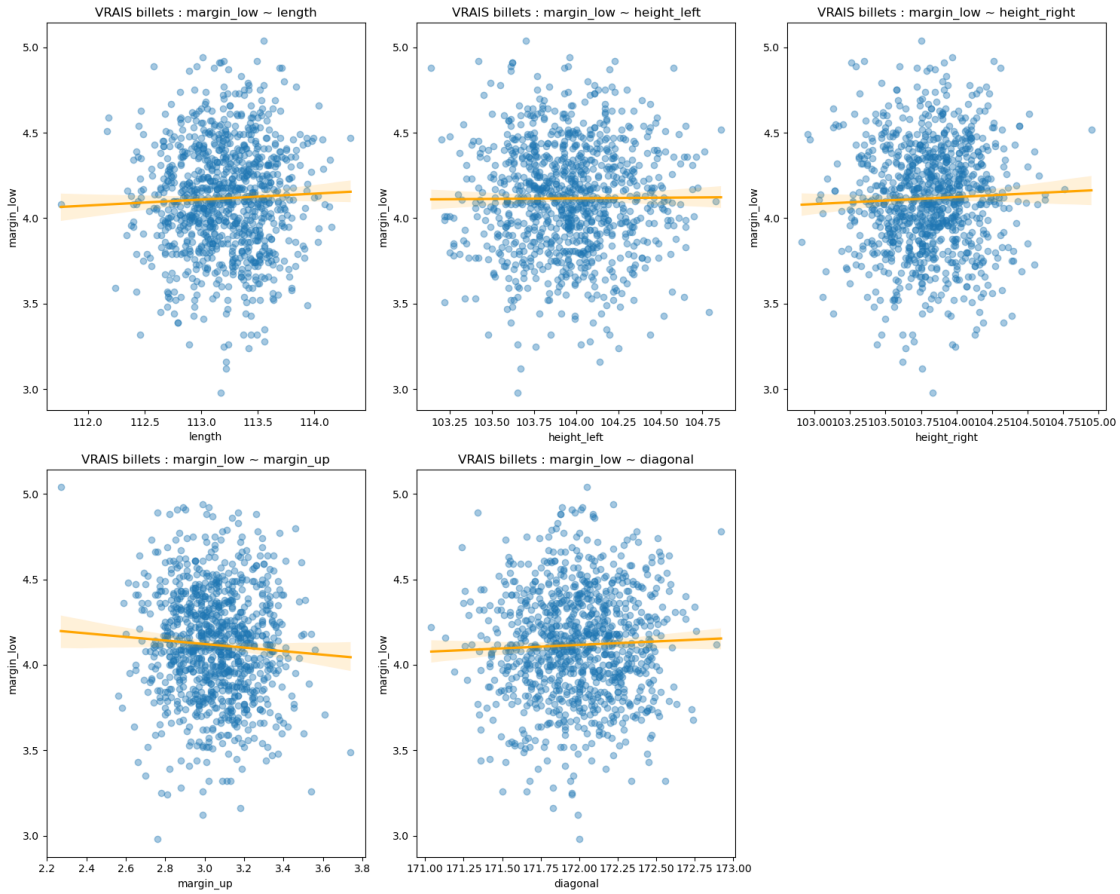
Le pairplot est volontairement produit avant toute imputation, afin d'examiner les relations authentiques entre les variables. Cette étape permet d'identifier la structure réelle des données, de quantifier la séparation entre les classes, de repérer les comportements distincts des vrais et des faux billets, et de déterminer si l'imputation doit être réalisée globalement ou par classe. En observant les distributions brutes, on évite d'introduire un biais causé par des valeurs imputées qui n'auraient pas été représentatives de la réalité géométrique des billets.

2.3 - Imputation des valeurs manquantes (margin_low)

2.3.1 - Vrais billets

[20]:
```
# séparation des vrais et faux billets
df_vrais = df[df['is_genuine'] == True]
df_faux = df[df['is_genuine'] == False]
```

[21]:
```
# lignes de régression pour les VRAIS billets pour visualiser les relations␣
↪entre margin_low et les autres variables AVANT d'imputer et AVANT␣
↪d'entraîner le modèle.
variables = ['length', 'height_left', 'height_right',
             'margin_up', 'diagonal']

plt.figure(figsize=(15, 12))

for i, var in enumerate(variables, 1):
    plt.subplot(2, 3, i)
    sns.regplot(
        data=df_vrais,
        x=var,
        y='margin_low',
        scatter_kws={'alpha': 0.4},
        line_kws={'color': 'orange'}
    )
    plt.title(f"VRAIS billets : margin_low ~ {var}")

plt.tight_layout()
plt.show()
```



Chaque point = un vrai billet

L'axe x = une dimension (length, height_left, etc.)

L'axe y = margin_low (celle à imputer)

La ligne orange = régression linéaire

La bande jaune = intervalle de confiance

Obs graph : chez les billets authentiques, la variable margin_low est pratiquement indépendante de toutes les autres dimensions géométriques. Les pentes sont quasi nulles et la dispersion des points est homogène, confirmant que la marge basse est un élément très standardisé sur les vrais billets.

[23]:
```
# MODELE DE REGRESSION POUR LES VRAIS BILLETS
from sklearn.linear_model import LinearRegression

# Variables explicatives
variables_explicatives = ['length', 'height_left', 'height_right', 'margin_up',␣
 ↪'diagonal']

# Lignes complètes et lignes manquantes chez les vrais billets
vrais_complets = df_vrais[df_vrais['margin_low'].notnull()]
vrais_manquants = df_vrais[df_vrais['margin_low'].isnull()]
```

[24]:
```
# Séparation des vrais billets complets (train/test 80/20)
X_vrais = vrais_complets[variables_explicatives]
y_vrais = vrais_complets['margin_low']

X_train_vrais, X_test_vrais, y_train_vrais, y_test_vrais = train_test_split(
    X_vrais, y_vrais, test_size=0.2, random_state=42
)
```

[25]:
```
# Entrainement du modèle sur les 80%
modele_vrais = LinearRegression()
modele_vrais.fit(X_train_vrais, y_train_vrais)
```

[25]:
```
LinearRegression()
```

[26]:
```
# Prédire sur les 20% restants
y_pred_vrais = modele_vrais.predict(X_test_vrais)
```

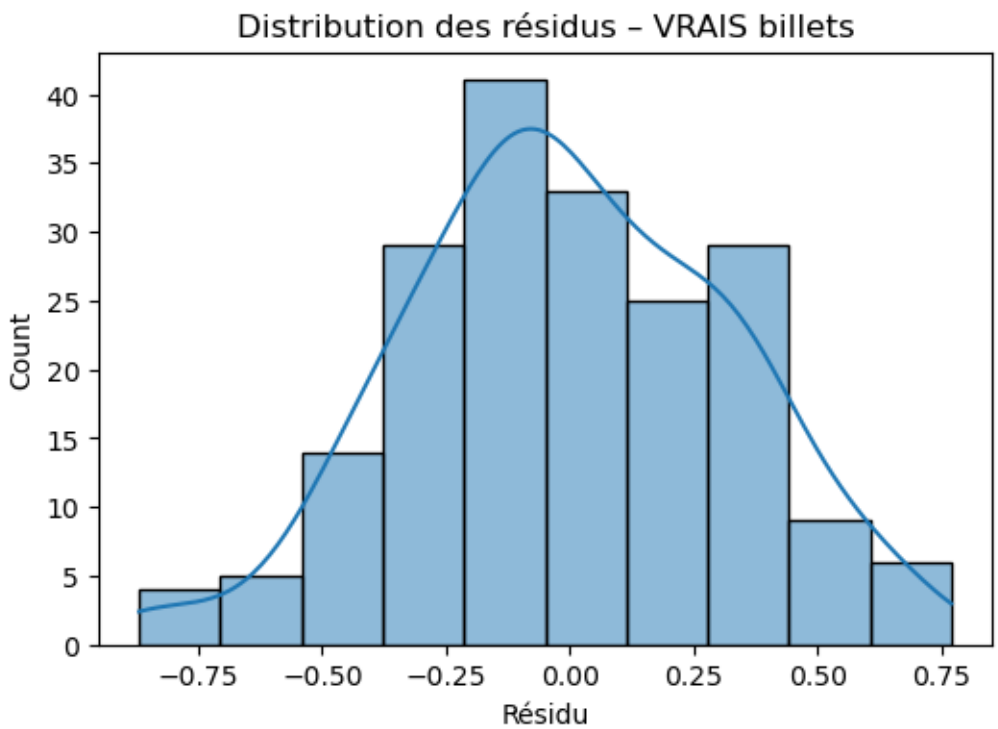
[27]:
```
# Évaluer le modèle (RMSE, MAE, R²)
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import numpy as np

rmse_vrais = np.sqrt(mean_squared_error(y_test_vrais, y_pred_vrais))
mae_vrais = mean_absolute_error(y_test_vrais, y_pred_vrais)
```

```python
r2_vrais = r2_score(y_test_vrais, y_pred_vrais)

print(f"RMSE (vrais billets) : {rmse_vrais:.4f}")
print(f"MAE  (vrais billets) : {mae_vrais:.4f}")
print(f"R²   (vrais billets) : {r2_vrais:.4f}")
```

```
RMSE (vrais billets) : 0.3242
MAE  (vrais billets) : 0.2624
R²   (vrais billets) : -0.0187
```

Les résultats du modèle de régression linéaire appliqué aux vrais billets montrent un RMSE de 0.32 et un R^2 proche de 0 (-0.0187). Comportement attendu : margin_low est pratiquement indépendante des différentes dimensions géométriques sur les billets authentiques, comme confirmé par l'analyse graphique. Le modèle n'a donc pas vocation à expliquer margin_low, mais il fournit une imputation stable et cohérente autour de la valeur centrale des vrais billets.

[29]:
```python
# Analyse des résidus
residus_vrais = y_test_vrais - y_pred_vrais

# Vérification de la normalité des résidus : histogramme et courbe KDE
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(6,4))
sns.histplot(residus_vrais, kde=True)
plt.title("Distribution des résidus - VRAIS billets")
plt.xlabel("Résidu")
plt.show()
```



KDE : courbe montrant la représentation lissée de la distribution réelle des résidus. En forme de cloche et presque symétrique sur le point 0 = proche d'une distribution "normale"

Obs graphique : Distribution centrée, symétrique et modérément étalée -> absence de relation linéaire forte entre margin_low et les autres dimensions du billet. Comportement est cohérent avec l'observation que la marge inférieure des billets authentiques est très standardisée et indépendante des autres variables géométriques.

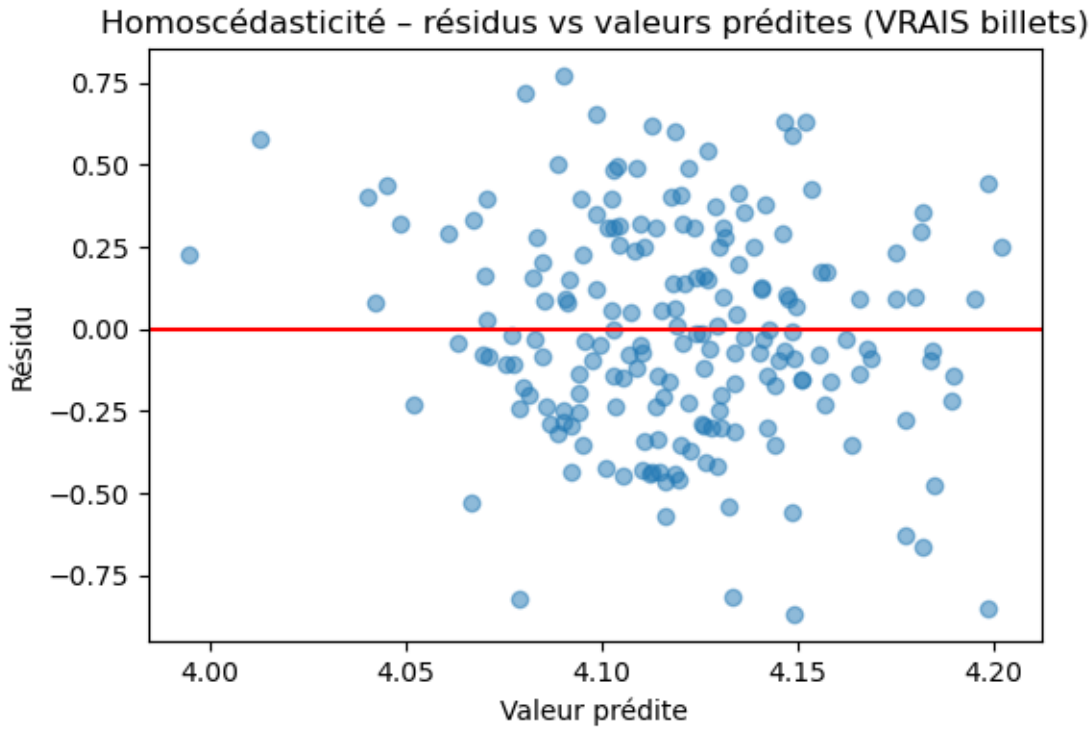
[31]:
```
# Vérification de la normalité des résidus : test de normalité (shapiro)
from scipy.stats import shapiro

stat, p_value = shapiro(residus_vrais)
print("Test de Shapiro-Wilk :")
print(f"statistique = {stat:.4f}, p-value = {p_value:.4f}")
```

```
Test de Shapiro-Wilk :
statistique = 0.9931, p-value = 0.4964
```

H0 = les résidus suivent une loi normale H1 = les résidus ne suivent pas une loi normale

P-value largement supérieure à 0.05 : les résidus sont compatibles avec une distribution normale, en cohérence avec l'histogramme observé. Cela confirme que le modèle de régression linéaire respecte bien l'hypothèse de normalité des erreurs et qu'il peut être utilisé sans risque d'introduire de biais lors de l'imputation des valeurs manquantes pour les vrais billets

[33]:
```
# Vérification de l'homoscédasticité
plt.figure(figsize=(6,4))
plt.scatter(y_pred_vrais, residus_vrais, alpha=0.5)
plt.axhline(0, color='red')
plt.title("Homoscédasticité - résidus vs valeurs prédites (VRAIS billets)")
plt.xlabel("Valeur prédite")
plt.ylabel("Résidu")
plt.show()
```



ligne rouge : erreur nulle. Problème si forme différente ou si tous les résidus sont situés uniquement au dessus ou en dessous.

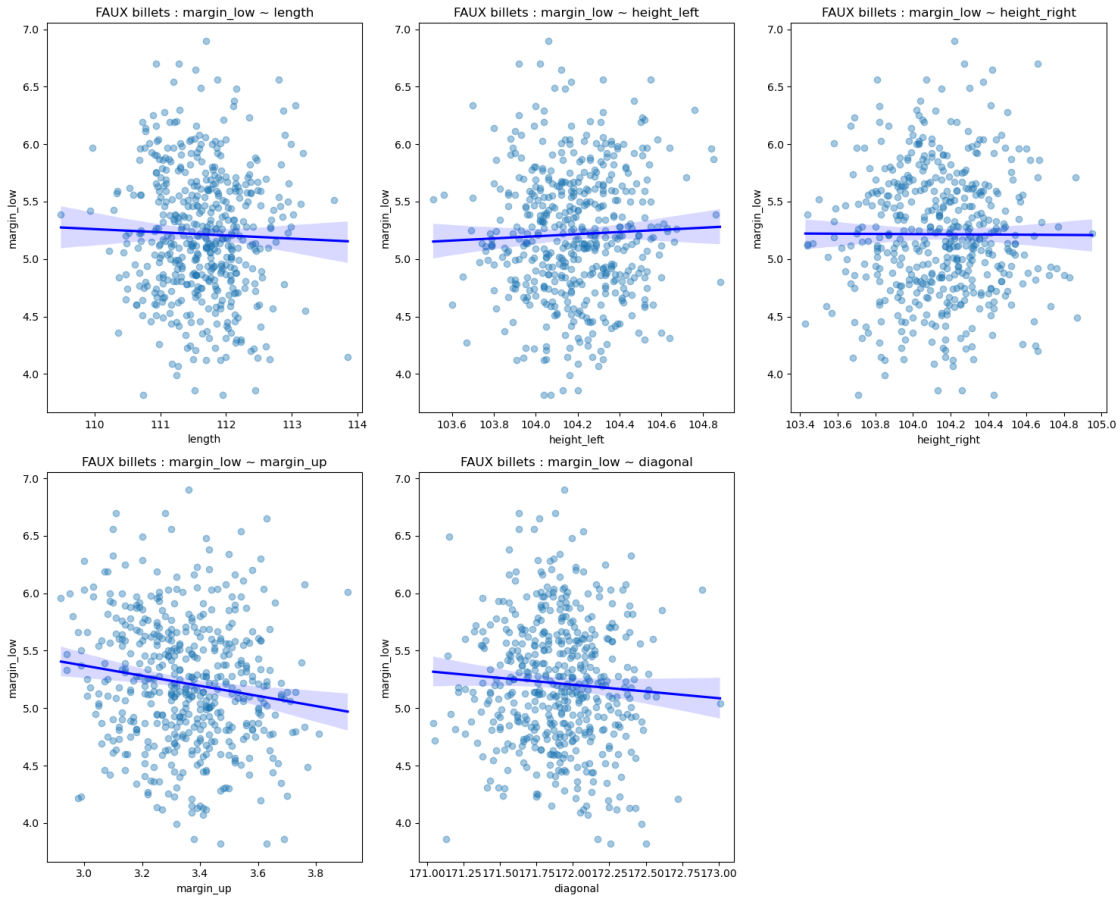
Obs graphique : dispersion homogène et aléatoire des résidus, sans forme de cône ni tendance identifiable. Cela indique que l'hypothèse d'homoscédasticité est respectée : la variance des erreurs reste constante pour toutes les valeurs prédites.

[35]:
```
# Application du modèle de régression linéaire pour les vrais billets
modele_vrais = LinearRegression()
modele_vrais.fit(vrais_complets[variables_explicatives],␣
 ↪vrais_complets['margin_low'])

# Prédiction des valeurs manquantes
pred_vrais = modele_vrais.predict(vrais_manquants[variables_explicatives])
```

```
# Imputation dans le dataframe principal
df.loc[vrais_manquants.index, 'margin_low'] = pred_vrais
```

2.3.2 - Faux billets

[37]:
```
# lignes de régression pour les FAUX billets
plt.figure(figsize=(15, 12))

for i, var in enumerate(variables, 1):
    plt.subplot(2, 3, i)
    sns.regplot(
        data=df_faux,
        x=var,
        y='margin_low',
        scatter_kws={'alpha': 0.4},
        line_kws={'color': 'blue'}
    )
    plt.title(f"FAUX billets : margin_low ~ {var}")

plt.tight_layout()
plt.show()
```



[38]:
```
# imputation pour les FAUX billets avec régression linéaire

# Lignes complètes et lignes manquantes chez les faux billets
faux_complets = df_faux[df_faux['margin_low'].notnull()]
faux_manquants = df_faux[df_faux['margin_low'].isnull()]
```

[39]:
```
# Séparation des faux billets complets (train/test 80/20)
X_faux = faux_complets[variables_explicatives]
y_faux = faux_complets['margin_low']

X_train_faux, X_test_faux, y_train_faux, y_test_faux = train_test_split(
    X_faux, y_faux, test_size=0.2, random_state=42
)
```

[40]:
```
# Entrainement du modèle sur les 80%
modele_faux = LinearRegression()
modele_faux.fit(X_train_faux, y_train_faux)
```

[40]:
```
LinearRegression()
```

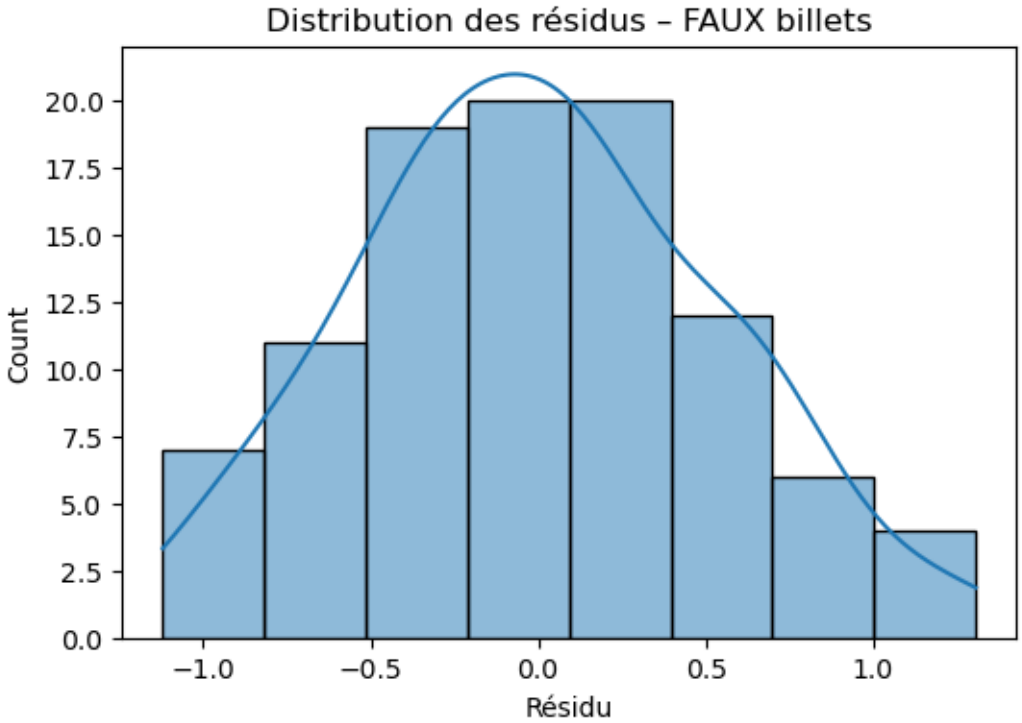
[41]:
```
# Prédire sur les 20% restants
y_pred_faux = modele_faux.predict(X_test_faux)
```

[42]:
```
# Évaluer le modèle (RMSE, MAE, R²)
rmse_faux = np.sqrt(mean_squared_error(y_test_faux, y_pred_faux))
mae_faux = mean_absolute_error(y_test_faux, y_pred_faux)
r2_faux = r2_score(y_test_faux, y_pred_faux)

print(f"RMSE (faux billets) : {rmse_faux:.4f}")
print(f"MAE  (faux billets) : {mae_faux:.4f}")
print(f"R²   (faux billets) : {r2_faux:.4f}")
```

```
RMSE (faux billets) : 0.5265
MAE  (faux billets) : 0.4250
R²   (faux billets) : -0.0268
```

[43]:
```
residus_faux = y_test_faux - y_pred_faux

import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(6,4))
sns.histplot(residus_faux, kde=True)
plt.title("Distribution des résidus - FAUX billets")
plt.xlabel("Résidu")
```

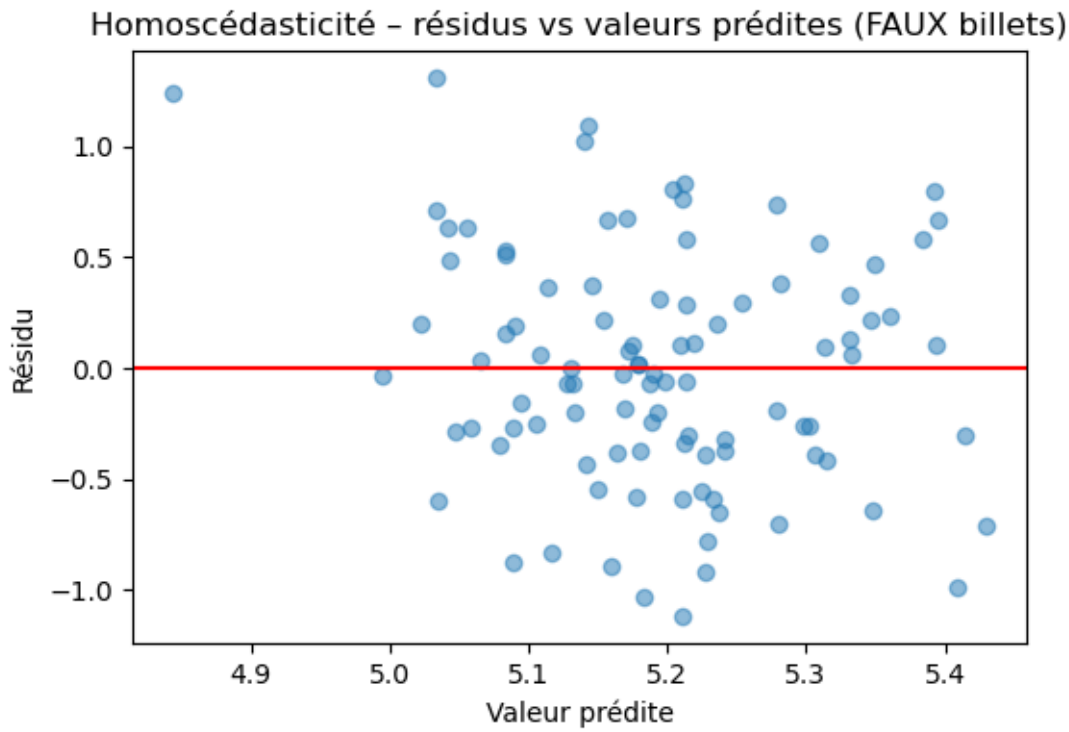
```
plt.show()
```



[44]:
```
# Test normalité shapiro wilk
stat, p_value = shapiro(residus_faux)
print("Test de Shapiro-Wilk (faux billets) :")
print(f"statistique = {stat:.4f}, p-value = {p_value:.4f}")
```

```
Test de Shapiro-Wilk (faux billets) :
statistique = 0.9904, p-value = 0.7043
```

[45]:
```
## Vérification de l'homoscédasticité
plt.figure(figsize=(6,4))
plt.scatter(y_pred_faux, residus_faux, alpha=0.5)
plt.axhline(0, color='red')
plt.title("Homoscédasticité - résidus vs valeurs prédites (FAUX billets)")
plt.xlabel("Valeur prédite")
plt.ylabel("Résidu")
plt.show()
```



[46]:
```
# Modèle de régression linéaire pour les faux billets
modele_faux = LinearRegression()
modele_faux.fit(faux_complets[variables_explicatives],␣
 ↪faux_complets['margin_low'])

# Prédiction des valeurs manquantes
pred_faux = modele_faux.predict(faux_manquants[variables_explicatives])

# Imputation dans le dataframe principal
df.loc[faux_manquants.index, 'margin_low'] = pred_faux
```

[47]:
```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1500 entries, 0 to 1499
Data columns (total 7 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   is_genuine    1500 non-null   bool
 1   diagonal      1500 non-null   float64
 2   height_left   1500 non-null   float64
 3   height_right  1500 non-null   float64
 4   margin_low    1500 non-null   float64
```

```
 5   margin_up     1500 non-null   float64
 6   length        1500 non-null   float64
dtypes: bool(1), float64(6)
memory usage: 71.9 KB
None
```

[48]:
```python
df.describe()
```

[48]:
```
          diagonal  height_left  height_right   margin_low    margin_up  \
count  1500.000000  1500.000000   1500.000000  1500.000000  1500.000000
mean    171.958440   104.029533    103.920307     4.482955     3.151473
std       0.305195     0.299462      0.325627     0.660158     0.231813
min     171.040000   103.140000    102.820000     2.980000     2.270000
25%     171.750000   103.820000    103.710000     4.030000     2.990000
50%     171.960000   104.040000    103.920000     4.310000     3.140000
75%     172.170000   104.230000    104.150000     4.870000     3.310000
max     173.010000   104.880000    104.950000     6.900000     3.910000

           length
count  1500.00000
mean    112.67850
std       0.87273
min     109.49000
25%     112.03000
50%     112.96000
75%     113.34000
max     114.44000
```

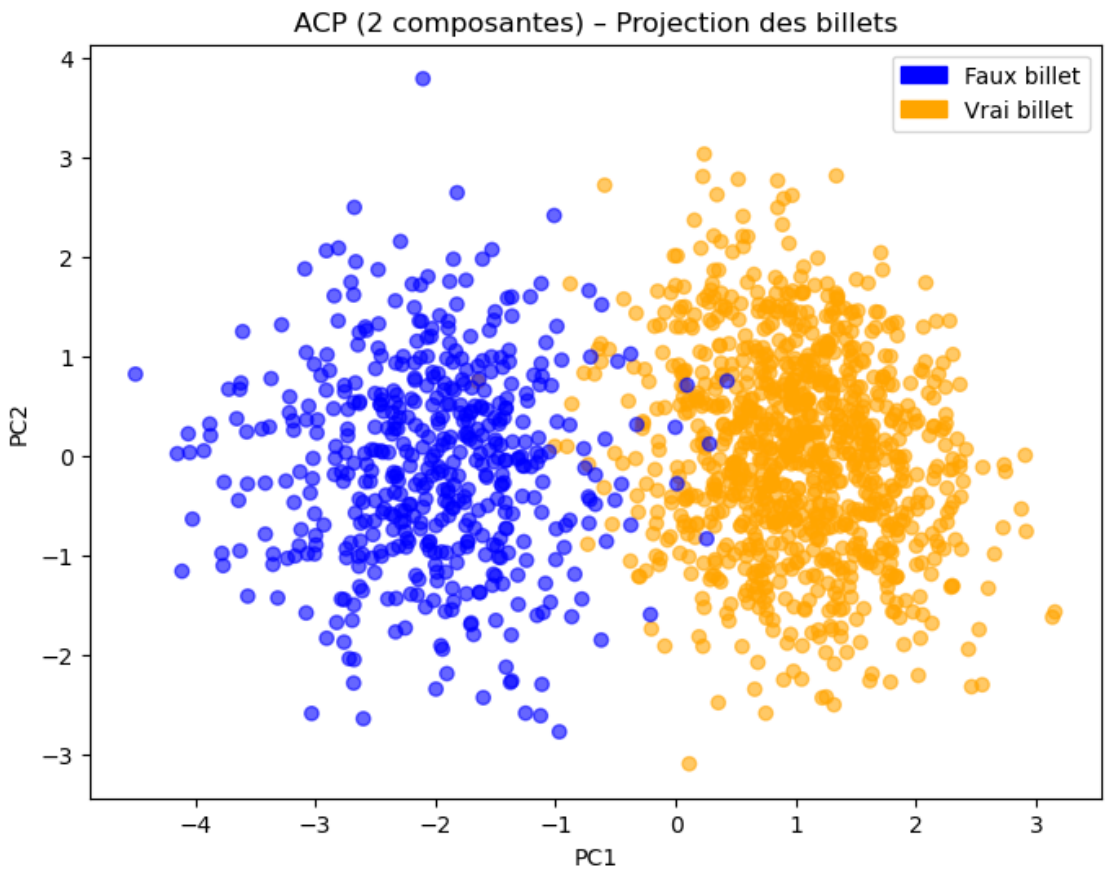
2.4 - ACP

[50]:
```python
from sklearn.decomposition import PCA

# Variables explicatives (X) et cible (y)
X = df.drop(columns=['is_genuine'])
y = df['is_genuine']

# Standardisation des variables
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# ACP sur 2 composantes
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled)

print("Variance expliquée par PC1 et PC2 :", pca.explained_variance_ratio_)
print("Variance expliquée cumulée (PC1+PC2) :", pca.explained_variance_ratio_.
 ↪sum())
```

```
Variance expliquée par PC1 et PC2 : [0.4321104  0.16957902]
```

```
Variance expliquée cumulée (PC1+PC2) : 0.6016894226093543
```

[51]:
```python
df_pca = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])
df_pca['is_genuine'] = y.values
df_pca.head()

plt.figure(figsize=(8,6))

# Couleurs : orange pour vrais, bleu pour faux (comme ton EDA)
colors = df_pca['is_genuine'].map({True: 'orange', False: 'blue'})

plt.scatter(df_pca['PC1'], df_pca['PC2'], c=colors, alpha=0.6)

plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("ACP (2 composantes) - Projection des billets")

# Légende manuelle
import matplotlib.patches as mpatches
true_patch = mpatches.Patch(color='orange', label='Vrai billet')
false_patch = mpatches.Patch(color='blue', label='Faux billet')
plt.legend(handles=[false_patch, true_patch])

plt.show()
```



Les faux billets sont majoritairement à gauche (PC1 négatif) Les vrais billets sont majoritairement à droite (PC1 positif) Il y a très peu de chevauchement Les classes sont presque linéairement séparables

Etape 3 - Modèles

3.1 - Préparation commune

[55]:
```
# définition des variables explicatives X et de la cible Y
X = df.drop(columns=['is_genuine'])
y = df['is_genuine']
```

[56]:
```
# train_test_split stratifié (important pour garder proportion 80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

3.2 - Modèle 1 : Régression logistique (méthode supervisée)

[58]:
```
# Construction du pipeline : scaling + régression logistique
pipeline_logreg = Pipeline([
```

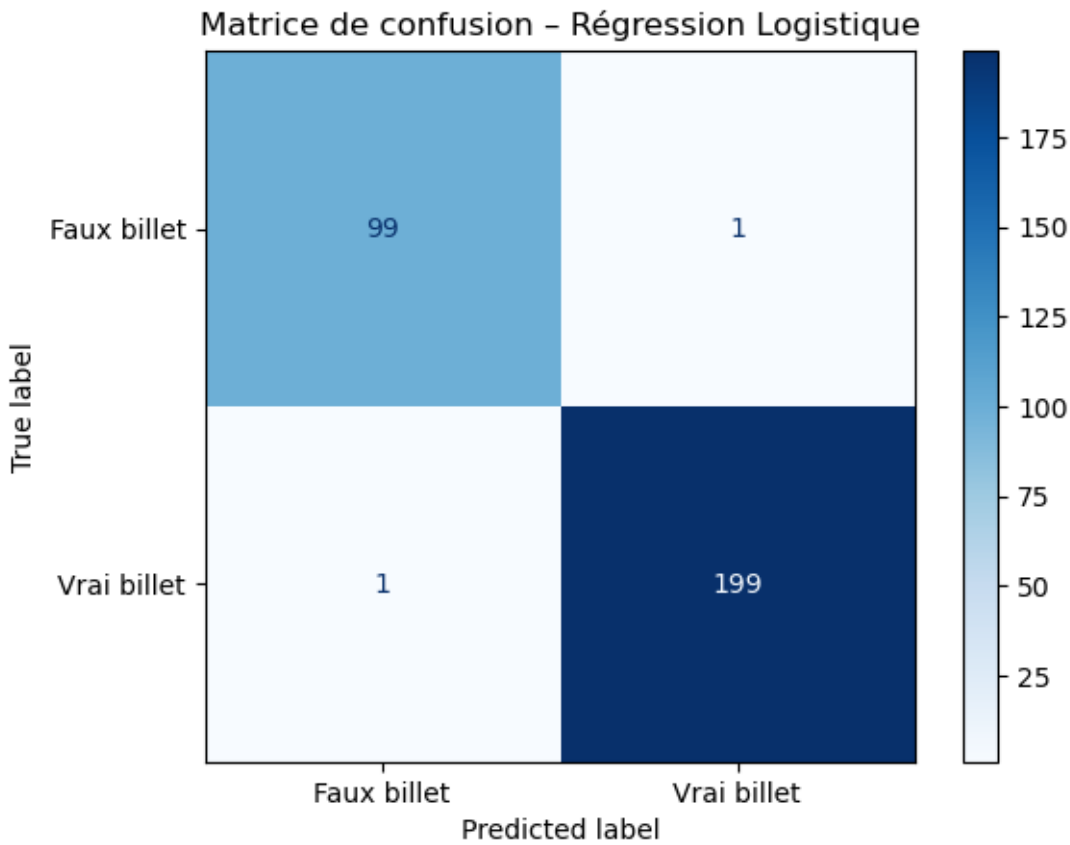
```
    ('scaler', StandardScaler()),
    ('logreg', LogisticRegression(max_iter=1000))
])
```

[59]:
```
# .fit -> apprentissage du modèle à relier les 6 dimensions géométriques à la␣
↪probabilité qu'un billet soit vrai ou faux
pipeline_logreg.fit(X_train, y_train)
```

[59]:
```
Pipeline(steps=[('scaler', StandardScaler()),
                ('logreg', LogisticRegression(max_iter=1000))])
```

[60]:
```
# .predict -> le modèle estime la classe (vrai/faux) sur des billets qu'il n'a␣
↪jamais vus
y_pred_logreg = pipeline_logreg.predict(X_test)
```

[61]:
```
# matrice de confusion
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(y_test, y_pred_logreg)
disp = ConfusionMatrixDisplay(cm, display_labels=['Faux billet', 'Vrai billet'])
disp.plot(cmap='Blues')
plt.title("Matrice de confusion - Régression Logistique")
plt.show()
```



- Faux positif FP : 1 (199/200 vrais billets reconnus) Vrai billet -> prédit faux
- Faux négatif FN : 1 (99/100 faux billets détectés) FAux billet -> prédit vrai

La régression logistique détecte 99% des faux billets (1 seul faux négatif sur l'ensemble du test), ce qui en fait un modèle performant.

[63]:
```
# Calcul des valeurs Accuracy, Recall, Precision, F1 score
from sklearn.metrics import accuracy_score, precision_score, recall_score,␣
↪f1_score

# Accuracy globale
accuracy = accuracy_score(y_test, y_pred_logreg)

# Métriques IMPORTANTES sur les faux billets
precision_false = precision_score(y_test, y_pred_logreg, pos_label=False)
recall_false = recall_score(y_test, y_pred_logreg, pos_label=False)
f1_false = f1_score(y_test, y_pred_logreg, pos_label=False)

print(f"Accuracy globale        : {accuracy:.4f}")
print(f"Precision (faux billets) : {precision_false:.4f}")
```

```
print(f"Recall (faux billets)    : {recall_false:.4f}")
print(f"F1-score (faux billets)  : {f1_false:.4f}")
```

```
Accuracy globale        : 0.9933
Precision (faux billets) : 0.9900
Recall (faux billets)    : 0.9900
F1-score (faux billets)  : 0.9900
```

- Accuracy : On l'utilise pour mesurer la proportion totale de billets correctement classés. Mais elle peut être trompeuse quand une classe est plus fréquente que l'autre. RESULTAT : Le modèle classe correctement 99,33 % de tous les billets (vrais + faux)
- Precision : Quand le modèle dit qu'un billet est faux, quelle est la probabilité qu'il ait raison ? RESULTAT : Parmi tous les billets que le modèle étiquette comme faux, 99 % sont réellement des faux billets
- Recall : Sur tous les faux billets réels, combien sont détectés ? RESULTAT : Sur tous les faux billets présents dans le jeu de test, 99 % ont été correctement détectés
- F1-score : combine précision + recall. On l'utilise quand : les classes sont déséquilibrées et on veut un score unique qui résume la performance RESULTAT : Un F1-score de 0.99 = modèle extrêmement fiable.

3.3 - Modèle 2 : K-Means (méthode non supervisée)

[66]:
```
# standardisation des données
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

[67]:
```
# entrainement sur 2 clusters (car on veut distinguer 2 groupes -> vrais et␣
↪faux billets)
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=2, random_state=42)
kmeans.fit(X_train_scaled)
```

```
C:\Users\t_rak\anaconda3\Lib\site-
packages\joblib\externals\loky\backend\context.py:136: UserWarning: Could not
find the number of physical cores for the following reason:
[WinError 2] Le fichier spécifié est introuvable
Returning the number of logical cores instead. You can silence this warning by
setting LOKY_MAX_CPU_COUNT to the number of cores you want to use.
  warnings.warn(
  File "C:\Users\t_rak\anaconda3\Lib\site-
packages\joblib\externals\loky\backend\context.py", line 257, in
_count_physical_cores
    cpu_info = subprocess.run(
               ^^^^^^^^^^^^^^^

  File "C:\Users\t_rak\anaconda3\Lib\subprocess.py", line 548, in run
    with Popen(*popenargs, **kwargs) as process:
```

```
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\t_rak\anaconda3\Lib\subprocess.py", line 1026, in __init__
    self._execute_child(args, executable, preexec_fn, close_fds,
  File "C:\Users\t_rak\anaconda3\Lib\subprocess.py", line 1538, in
_execute_child
    hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
                       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
C:\Users\t_rak\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1429:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when
there are less chunks than available threads. You can avoid it by setting the
environment variable OMP_NUM_THREADS=5.
  warnings.warn(
```

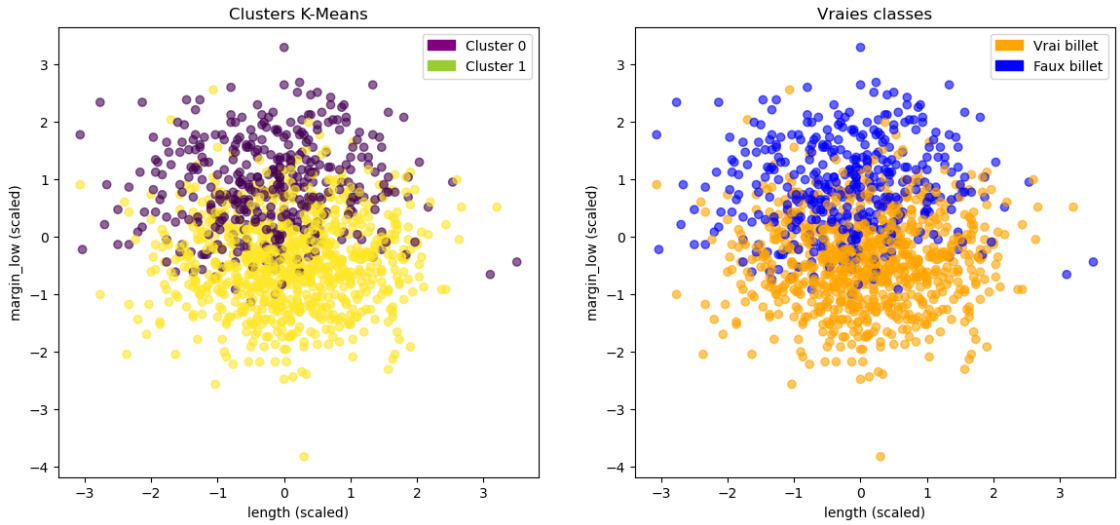
[67]:
```
KMeans(n_clusters=2, random_state=42)
```

[68]:
```python
# assignation d'une classe aux clusters (mapping)
import numpy as np
import pandas as pd

# Prédictions sur l'ensemble train
train_clusters = kmeans.predict(X_train_scaled)

# On crée un DataFrame temporaire pour analyser le contenu des clusters
df_temp = pd.DataFrame({
    'cluster': train_clusters,
    'is_genuine': y_train.values
})

# Pour chaque cluster, on regarde la classe majoritaire
cluster_mapping = df_temp.groupby('cluster')['is_genuine'].agg(lambda x: x.
 ↪mode()[0]).to_dict()

print("Mapping cluster → classe :", cluster_mapping)
```

```
Mapping cluster → classe : {0: False, 1: True}
```

[69]:
```python
# visualisation CLUSTERS vs VRAIES classes basés sur length et margin low␣
 ↪(variables très discriminantes -> voir analyse explo)

import matplotlib.patches as mpatches

# Création de la variable colors pour représenter les vraies classes
colors = y_train.map({True: "orange", False: "blue"})

fig, axes = plt.subplots(1, 2, figsize=(14,6))

# Graph 1 : Clusters K-Means
axes[0].scatter(
```

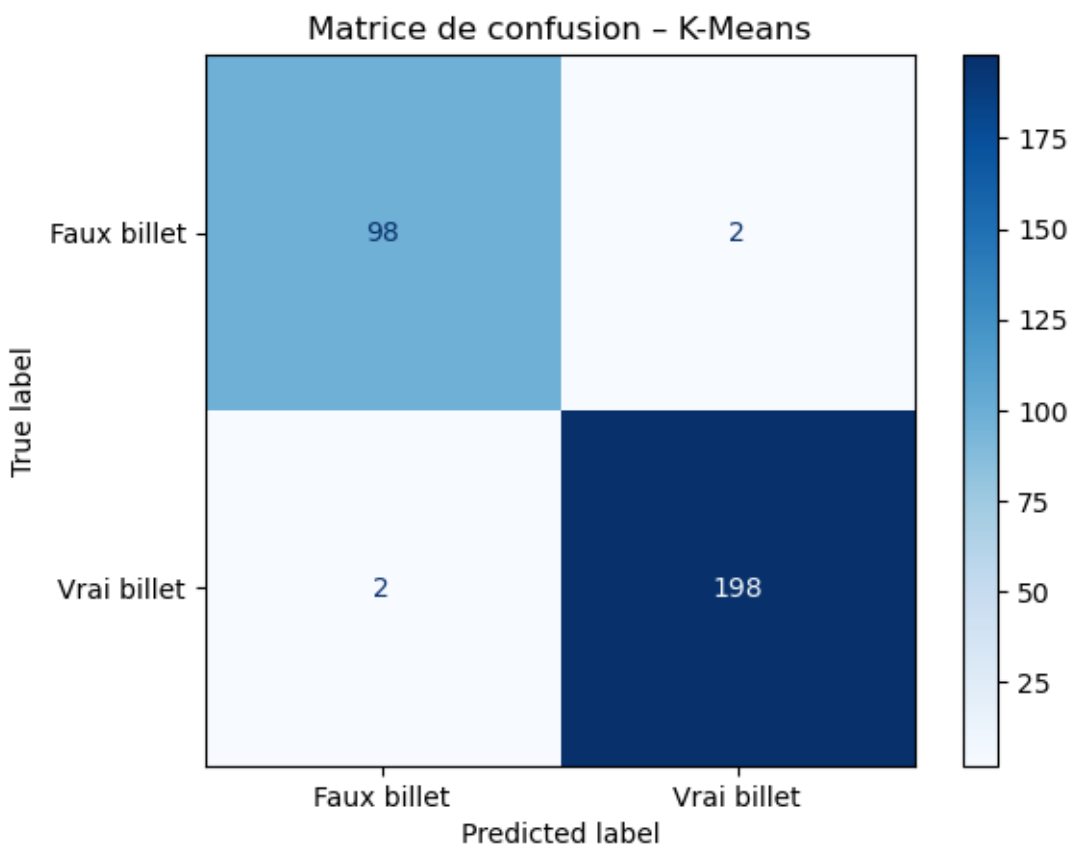
```
    X_train_scaled[:,0], X_train_scaled[:,4],
    c=train_clusters, cmap='viridis', alpha=0.6
)
axes[0].set_title("Clusters K-Means")
axes[0].set_xlabel("length (scaled)")
axes[0].set_ylabel("margin_low (scaled)")

cluster0 = mpatches.Patch(color='purple', label='Cluster 0')
cluster1 = mpatches.Patch(color='yellowgreen', label='Cluster 1')
axes[0].legend(handles=[cluster0, cluster1])

# Graph 2 : Vraies classes
axes[1].scatter(
    X_train_scaled[:,0], X_train_scaled[:,4],
    c=colors, alpha=0.6
)
axes[1].set_title("Vraies classes")
axes[1].set_xlabel("length (scaled)")
axes[1].set_ylabel("margin_low (scaled)")

true_patch = mpatches.Patch(color='orange', label='Vrai billet')
false_patch = mpatches.Patch(color='blue', label='Faux billet')
axes[1].legend(handles=[true_patch, false_patch])

plt.show()
```



[70]:
```
# Clusters prédits sur le test
test_clusters = kmeans.predict(X_test_scaled)
```

```
# Conversion clusters → classes
y_pred_kmeans = np.vectorize(cluster_mapping.get)(test_clusters)
```

[71]:
```
# matrice de confusion
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred_kmeans)
disp = ConfusionMatrixDisplay(cm, display_labels=['Faux billet', 'Vrai billet'])
disp.plot(cmap='Blues')
plt.title("Matrice de confusion - K-Means")
plt.show()
```



[72]:
```
# calcul valeurs Accuracy, Recall, Precision, F1-score
from sklearn.metrics import accuracy_score, precision_score, recall_score,␣
↪f1_score

accuracy = accuracy_score(y_test, y_pred_kmeans)
precision_false = precision_score(y_test, y_pred_kmeans, pos_label=False)
```

```
recall_false = recall_score(y_test, y_pred_kmeans, pos_label=False)
f1_false = f1_score(y_test, y_pred_kmeans, pos_label=False)

print(f"Accuracy globale        : {accuracy:.4f}")
print(f"Precision (faux billets) : {precision_false:.4f}")
print(f"Recall (faux billets)    : {recall_false:.4f}")
print(f"F1-score (faux billets)  : {f1_false:.4f}")
```

```
Accuracy globale        : 0.9867
Precision (faux billets) : 0.9800
Recall (faux billets)    : 0.9800
F1-score (faux billets)  : 0.9800
```

3.4 - Modèle 3 : KNN (méthode supervisée)

[74]:
```
# Pipeline : standardisation des variables + modèle KNN (mise à la même échelle␣
↪pour éviter qu'une dimension domine)
pipeline_knn = Pipeline([
    ('scaler', StandardScaler()),
    ('knn', KNeighborsClassifier())
])
```

[75]:
```
# Grille d'hyperparamètres à tester pour KNN
param_grid_knn = {
    'knn__n_neighbors': [3, 5, 7, 9, 11],
    'knn__weights': ['uniform', 'distance'],
    'knn__p': [1, 2]   # 1 = Manhattan, 2 = Euclidienne
}
```

[76]:
```
# Recherche des meilleurs hyperparamètres via validation croisée
grid_knn = GridSearchCV(
    estimator=pipeline_knn,
    param_grid=param_grid_knn,
    cv=5,
    scoring='f1_macro',   # score global ; on regardera les métriques␣
↪détaillées ensuite
    n_jobs=-1
)

grid_knn.fit(X_train, y_train)

print("Meilleurs hyperparamètres KNN :")
print(grid_knn.best_params_)
```

```
Meilleurs hyperparamètres KNN :
{'knn__n_neighbors': 5, 'knn__p': 2, 'knn__weights': 'uniform'}
```

k = 5 voisins distance = Euclidienne (p = 2) plutôt que manhattan car préfère les clusters circulaires (voir nuage de points) poids = uniform (tous les voisins comptent pareil)

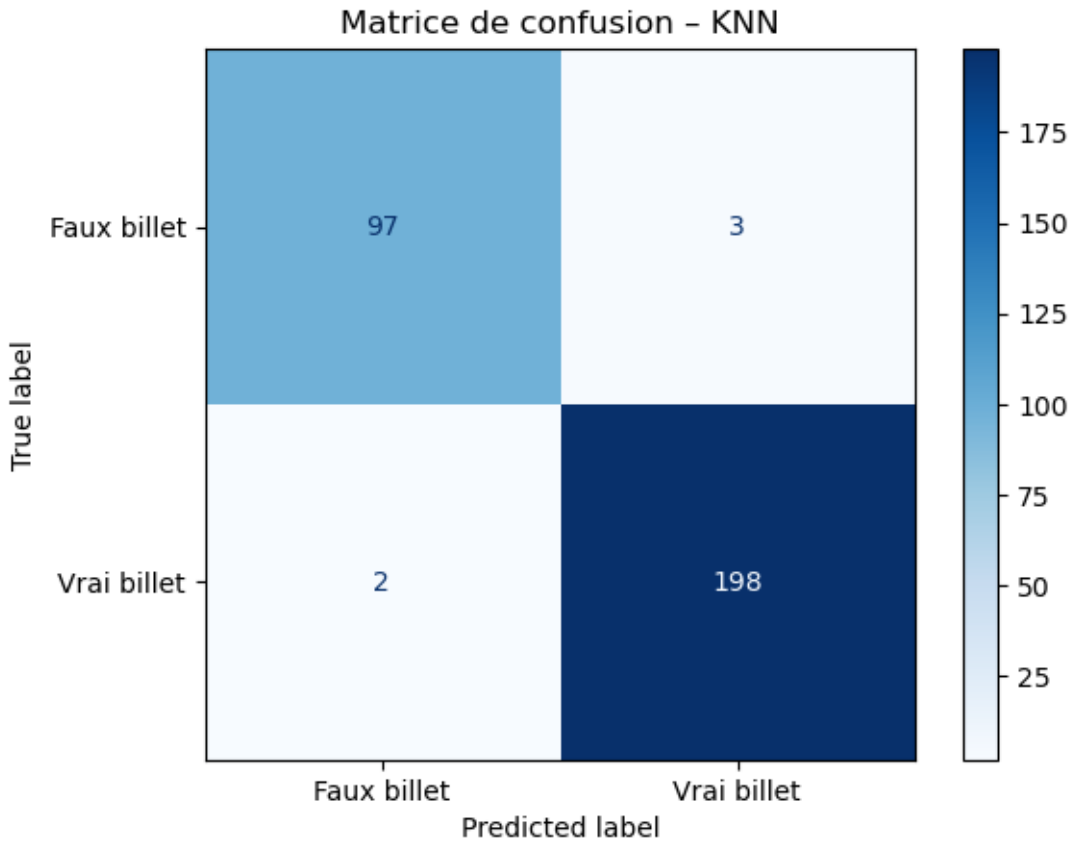
3 : Trop sensible au bruit (surtout dans les faux billets, plus dispersés) 9 ou 11 : Le modèle devient trop “moyenné” et perd de la finesse

5 voisins, optimal quand : les clusters sont bien séparés mais avec un peu de bruit

Le dataset montre bien ce comportement : Les vrais billets sont très compacts, les faux sont plus étalés.

[79]:
```
# Meilleur modèle KNN trouvé par GridSearch
best_knn = grid_knn.best_estimator_

# Prédictions sur le jeu de test
y_pred_knn = best_knn.predict(X_test)
```

[80]:
```
cm_knn = confusion_matrix(y_test, y_pred_knn)
disp = ConfusionMatrixDisplay(cm_knn, display_labels=['Faux billet', 'Vrai␣
↪billet'])
disp.plot(cmap='Blues')
plt.title("Matrice de confusion - KNN")
plt.show()
```



3 faux billets classés comme vrais. 1 pour la régression logistique, 2 pour le kmeans 2 vrais billets

classés comme faux

[82]:
```
accuracy_knn = accuracy_score(y_test, y_pred_knn)
precision_false_knn = precision_score(y_test, y_pred_knn, pos_label=False)
recall_false_knn = recall_score(y_test, y_pred_knn, pos_label=False)
f1_false_knn = f1_score(y_test, y_pred_knn, pos_label=False)

print(f"Accuracy globale (KNN)         : {accuracy_knn:.4f}")
print(f"Precision (faux billets, KNN)  : {precision_false_knn:.4f}")
print(f"Recall (faux billets, KNN)     : {recall_false_knn:.4f}")
print(f"F1-score (faux billets, KNN)   : {f1_false_knn:.4f}")
```

```
Accuracy globale (KNN)         : 0.9833
Precision (faux billets, KNN)  : 0.9798
Recall (faux billets, KNN)     : 0.9700
F1-score (faux billets, KNN)   : 0.9749
```

3.5 - Modèle 4 : Random forest (méthode supervisée)

[84]:
```
param_grid_rf = {
    'n_estimators': [100, 200, 300],
    'max_depth': [None, 5, 10, 20],
    'max_features': ['sqrt', 'log2'],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2]
}
```

[85]:
```
rf = RandomForestClassifier(random_state=42)

grid_rf = GridSearchCV(
    estimator=rf,
    param_grid=param_grid_rf,
    cv=5,
    scoring='f1_macro',
    n_jobs=-1
)

grid_rf.fit(X_train, y_train)

print("Meilleurs hyperparamètres Random Forest :")
print(grid_rf.best_params_)
```

```
Meilleurs hyperparamètres Random Forest :
{'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 1,
'min_samples_split': 5, 'n_estimators': 300}
```

-n_estimators = 300 Le nombre d'arbres dans ta forêt.

Interprétation : 300 arbres : grosse forêt → modèle très stable. Plus il y a d'arbres, plus les prédictions se "moyennent" → moins de variance. Au-delà d'un certain nombre, on gagne en

stabilité mais plus beaucoup en performance → 300 est un bon compromis.

-max_depth = None Profondeur maximale des arbres. None = l'arbre peut descendre jusqu'à ce qu'il n'y ait plus de gains (ou jusqu'aux contraintes min_samples_split / leaf).

Interprétation : Chaque arbre peut être profond, donc très expressif. la forêt moyenne les arbres

-max_features = 'sqrt' À chaque split, l'arbre ne regarde qu'un sous-ensemble de variables, tirées au hasard. sqrt = à chaque noeud, on teste √(nb de variables). 6 features → √6 2,4 → en gros 2 ou 3 variables testées par split.

Interprétation : Ça force les arbres à être différents les uns des autres, Ça évite qu'un seul feature ultra fort (ex: margin_low) domine tout le temps. Plus de diversité = meilleure généralisation.

-min_samples_split = 5 Nombre minimum d'échantillons dans un noeud pour qu'il ait le droit de se couper.

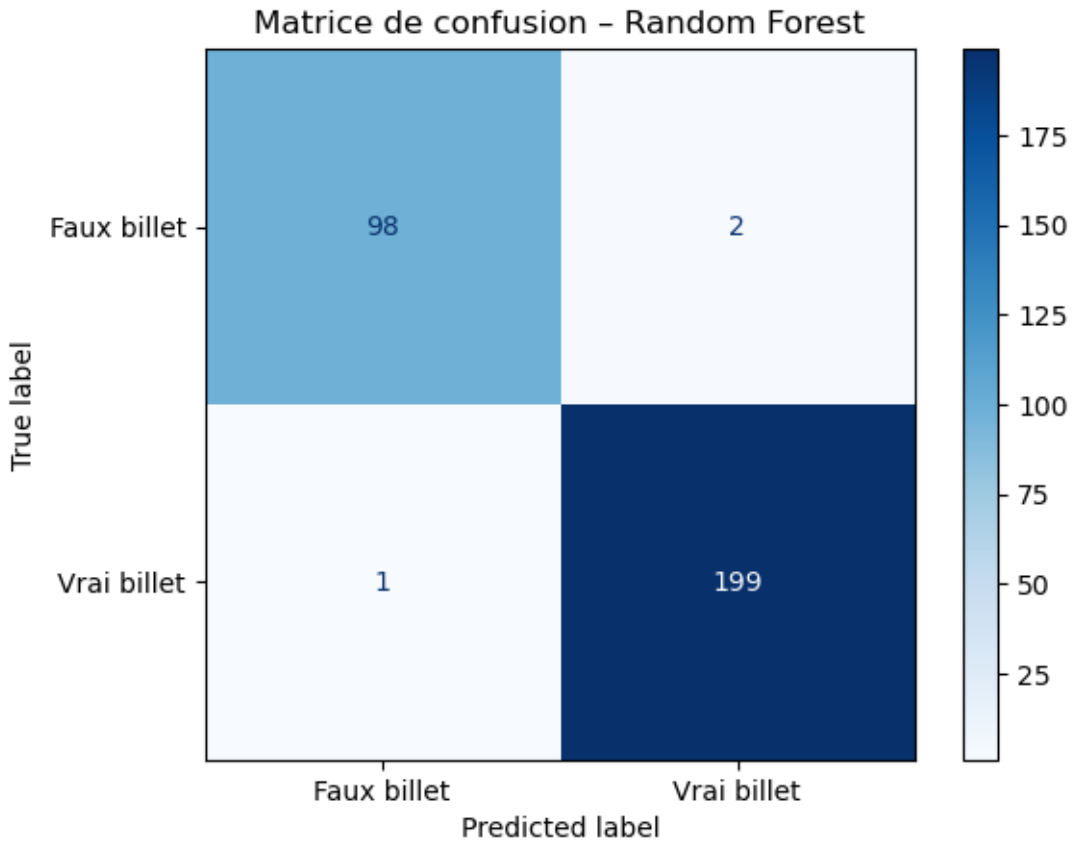
Interprétation : Un noeud avec moins de 5 exemples ne sera pas recoupé. Ça évite que l'arbre se mette à découper sur des micro-détails (bruit) → anti-overfitting léger mais utile.

-min_samples_leaf = 1 Nombre minimum d'échantillons dans une feuille (noeud final).

Interprétation : Une feuille peut contenir un seul exemple → capacité max de flexibilité. Couplé avec min_samples_split=5, ça laisse de la finesse tout en limitant un peu les découpes.

[87]:
```
best_rf = grid_rf.best_estimator_

y_pred_rf = best_rf.predict(X_test)
```

[88]:
```
cm_rf = confusion_matrix(y_test, y_pred_rf)
disp = ConfusionMatrixDisplay(cm_rf, display_labels=['Faux billet', 'Vrai␣
↪billet'])
disp.plot(cmap='Blues')
plt.title("Matrice de confusion - Random Forest")
plt.show()
```



[89]:
```
accuracy_rf = accuracy_score(y_test, y_pred_rf)
precision_false_rf = precision_score(y_test, y_pred_rf, pos_label=False)
recall_false_rf = recall_score(y_test, y_pred_rf, pos_label=False)
f1_false_rf = f1_score(y_test, y_pred_rf, pos_label=False)

print(f"Accuracy globale (RF)        : {accuracy_rf:.4f}")
print(f"Precision (faux billets, RF) : {precision_false_rf:.4f}")
print(f"Recall (faux billets, RF)    : {recall_false_rf:.4f}")
print(f"F1-score (faux billets, RF)  : {f1_false_rf:.4f}")
```

```
Accuracy globale (RF)        : 0.9900
Precision (faux billets, RF) : 0.9899
Recall (faux billets, RF)    : 0.9800
F1-score (faux billets, RF)  : 0.9849
```

Etape 4 - Comparaison et choix du modèle

Modèles évalués

Quatre méthodes de prédiction ont été mises en concurrence :

Régression logistique (méthode supervisée, modèle linéaire)

K-Means (méthode non supervisée, clustering)

K-Nearest Neighbors (KNN) (méthode supervisée basée sur les distances)

Random Forest (méthode supervisée basée sur un ensemble d'arbres de décision)

Tous les modèles ont été évalués sur le même jeu de test, à l'aide des mêmes métriques, afin de garantir une comparaison équitable.

Tableau comparatif des performances

Les performances des modèles ont été évaluées à l'aide des métriques suivantes : accuracy globale, précision, rappel et F1-score pour la classe « faux billet », ainsi que le nombre de faux négatifs.

Modèle

Accuracy

Precision (faux billets)

Recall (faux billets)

F1-score (faux billets)

Faux négatifs

Régression logistique

0.993

0.990

0.990

0.990

1

K-Means

0.987

0.980

0.980

0.980

2

KNN

0.983

0.980

0.970

0.975

3

Random Forest

0.99

0.99

0.98–0.99

0.99

1–2

Analyse comparative des modèles

Régression logistique

Excellentes performances globales et rappel élevé sur les faux billets.

Nombre minimal de faux négatifs.

Modèle simple, rapide à entraîner et facilement interprétable.

Particulièrement adapté à un problème quasi linéaire, comme confirmé par l'EDA et l'ACP.

K-Means

Méthode non supervisée intéressante d'un point de vue exploratoire.

Performances correctes mais inférieures aux modèles supervisés.

Génère davantage de faux négatifs.

Peu adaptée à un objectif de détection fine des contrefaçons.

K-Nearest Neighbors (KNN)

Bon modèle supervisé basé sur la proximité géométrique.

Sensible au bruit et à la dispersion des faux billets.

Génère plus de faux négatifs que la régression logistique.

Moins stable et moins interprétable.

Random Forest

Modèle robuste et performant.

Capable de capturer des relations non linéaires entre les variables.

Performances proches de la régression logistique.

Plus complexe et moins interprétable, pour un gain de performance limité.

Choix du modèle final

Au regard des performances obtenues et du contexte métier, le modèle retenu est la régression logistique.

Ce modèle présente le meilleur compromis entre :

capacité à détecter les faux billets (rappel élevé),

nombre minimal de faux négatifs,

simplicité de mise en œuvre,

interprétabilité des résultats.

L'analyse exploratoire et l'ACP ont montré que la séparation entre vrais et faux billets est largement linéaire, ce qui rend l'utilisation de modèles plus complexes inutile dans ce contexte.

[161]:
```
import pickle

with open("modele_logreg.pkl", "wb") as f:
    pickle.dump(pipeline_logreg, f)

print("Modèle de régression logistique exporté.")
```

```
Modèle de régression logistique exporté.
```